

Análise de Desempenho e Confiabilidade em Camadas de Transporte: TCP vs. R-UDP

João Vitor Santana Lima¹

¹Universidade Federal do Piauí – Picos – PI – Brasil

joao.vitor.1@ufpi.edu.br

- **Link do vídeo:** <https://www.youtube.com/watch?v=13jUMfOo2Oo>
- **Link do repositório:** <https://github.com/jvitorsantana/trabalho-1-redes-ii>

Resumo. *Este trabalho apresenta a implementação e a comparação de dois sistemas de transferência de arquivos escritos em Python: um que usa o protocolo TCP nativo e outro que usa o UDP com uma camada de confiabilidade desenvolvida para este trabalho, o R-UDP. O R-UDP foi construído com o método Stop-and-Wait, com números de sequência, confirmações (ACKs), tempo limite com retransmissão e verificação de integridade por CRC-32 em cada bloco. Os testes foram feitos em contêineres Docker, simulando redes ruins com o `tc` netem em três cenários: 0%, 5% e 10% de perda de pacotes, combinados com atrasos de 10, 50 e 100 ms. A conferência entre os dados medidos pela aplicação e os arquivos `.pcap` capturados com o `tcpdump` mostrou que os números batem. Os resultados mostram que o TCP teve uma vazão muito maior em todos os cenários, enquanto o R-UDP entregou os arquivos sem erros mesmo com muita perda, mas com uma vazão muito baixa por causa do funcionamento do Stop-and-Wait.*

1. Introdução

Quando dois computadores trocam dados pela rede, quem cuida da comunicação entre os programas é a camada de transporte. Os dois protocolos mais usados nessa camada são o TCP e o UDP, e eles funcionam de formas bem diferentes.

O TCP (*Transmission Control Protocol*) é um protocolo confiável. Ele garante que todos os dados cheguem ao destino, na ordem certa e sem erros. Para isso, ele cria uma conexão antes de enviar os dados, reenvia o que se perde e controla a velocidade do envio. Tudo isso já vem pronto no sistema operacional, mas tem um custo: mais processamento e mais informações de controle trafegando na rede.

O UDP (*User Datagram Protocol*) é o oposto. Ele apenas envia os dados e não se preocupa se eles chegaram ou não. Não existe conexão, não existe reenvio e não existe controle de ordem. Por isso o UDP é mais simples e mais rápido, mas, sozinho, ele não serve para transferir um arquivo sem risco de perder pedaços.

Daí vem a motivação deste trabalho: é possível pegar o UDP, que é simples, e adicionar em cima dele, na própria aplicação, os mecanismos que faltam para torná-lo confiável. Esse protocolo desenvolvido é chamado de R-UDP (*Reliable UDP*, ou UDP confiável). Comparar o R-UDP com o TCP ajuda a entender o que o TCP faz por baixo dos panos e qual é o preço de garantir a entrega dos dados.

1.1. Objetivos

O objetivo geral deste trabalho é implementar e comparar dois sistemas de transferência de arquivos, um usando TCP e outro usando R-UDP, medindo o desempenho de cada um sob diferentes condições de rede.

Os objetivos específicos são:

- Implementar controle de fluxo e de erros sobre o UDP, usando o método Stop-and-Wait.
- Criar um protocolo de aplicação com cabeçalhos próprios e um campo de autenticação baseado em hash.
- Simular condições ruins de rede (perda e atraso) dentro do Docker.
- Conferir os dados da aplicação com o que foi capturado pelo `tcpdump`.
- Gerar uma análise estatística comparando TCP e R-UDP em cada cenário.

2. Fundamentação Teórica

2.1. Protocolo TCP

O TCP é um protocolo orientado à conexão. Antes de enviar qualquer dado, o cliente e o servidor fazem um aperto de mãos (o *three-way handshake*) para abrir a conexão. A partir daí, o TCP cuida de tudo sozinho.

A confiabilidade do TCP vem de três ideias principais. Primeiro, cada byte enviado tem um número de sequência, então o receptor sabe montar os dados na ordem certa. Segundo, o receptor envia confirmações (ACKs) avisando o que já recebeu, e o que não foi confirmado é reenviado. Terceiro, o TCP usa uma soma de verificação para detectar dados corrompidos.

Além disso, o TCP tem controle de fluxo e controle de congestionamento. O controle de fluxo evita que o emissor mande dados rápido demais para um receptor lento. O controle de congestionamento ajusta a velocidade de envio de acordo com o estado da rede, diminuindo o ritmo quando percebe perdas e aumentando aos poucos quando a rede está livre.

2.2. Protocolo UDP

O UDP é um protocolo sem conexão. Ele pega os dados, coloca um cabeçalho pequeno de apenas 8 bytes e envia, sem garantir nada. Não há aperto de mãos, não há confirmação e não há reenvio.

A vantagem do UDP é a simplicidade e a baixa sobrecarga, o que o torna rápido e leve. Ele é bom para coisas como vídeo ao vivo e jogos, onde perder um pacote de vez em quando é melhor do que esperar por um reenvio. A limitação é justamente a falta de garantias: se um pacote se perde ou chega fora de ordem, o UDP não faz nada a respeito. Quem precisar dessas garantias tem que implementá-las por conta própria.

2.3. Reliable UDP (R-UDP)

O R-UDP é a ideia de construir, em cima do UDP, os mecanismos de confiabilidade que ele não tem. Como o UDP só entrega e recebe pacotes, cabe ao programa da aplicação

numerar os pacotes, confirmar o que chegou, reenviar o que se perdeu e checar se os dados vieram corretos.

Existem várias estratégias para fazer isso. As mais conhecidas são o Stop-and-Wait, o Go-Back-N e o Selective Repeat. As duas últimas trabalham com uma janela, ou seja, enviam vários pacotes antes de esperar confirmação, o que deixa a transferência mais rápida, porém com um controle mais complicado.

Neste trabalho foi usado o Stop-and-Wait, que é o mais simples de todos. A regra é direta: o emissor manda um pacote e fica parado esperando a confirmação dele. Só depois que o ACK chega é que o próximo pacote é enviado. Se a confirmação não chegar dentro de um tempo limite, o mesmo pacote é reenviado. Esse método é fácil de entender e de implementar, mas, como mostram os resultados, ele é lento, pois fica a maior parte do tempo esperando, sem aproveitar bem a rede.

3. Metodologia

3.1. Ambiente Experimental

Os testes foram montados com dois contêineres Docker baseados na imagem `ubuntu:24.04`, um para o servidor e outro para o cliente. Os dois ficam ligados por uma rede Docker do tipo *bridge*, criada só para o trabalho. O contêiner do servidor recebe a permissão `NET_ADMIN`, que é o que permite alterar as configurações de rede com o `tc`.

A organização do código segue o modelo cliente-servidor. O servidor fica escutando em uma porta, recebe os blocos do arquivo, confere cada um e grava o resultado em disco. O cliente lê o arquivo, divide em blocos e envia cada bloco seguindo as regras do protocolo escolhido (TCP ou R-UDP). A porta usada foi a 25565. A máquina onde tudo foi executado está descrita na Tabela 1.

Tabela 1. Especificações da máquina de testes

Componente	Especificação
Sistema Operacional	Windows 11 Pro (Build 26200.8457)
Processador	AMD Ryzen 9 7950X
Memória RAM	32 GB
Versão do Docker	29.4.3
Versão do Python	3.12

3.2. Ferramentas Utilizadas

Quatro ferramentas principais foram usadas no trabalho:

- **Docker:** cria os contêineres isolados que simulam duas máquinas diferentes na mesma rede, garantindo que o teste seja sempre igual.
- **tc qdisc (netem):** comando do Linux que altera a rede de propósito, adicionando atraso e perda de pacotes para simular uma rede ruim.
- **tcpdump e Wireshark:** o `tcpdump` captura todos os pacotes que passam pela rede e salva em um arquivo `.pcap`; o Wireshark é usado depois para abrir esse arquivo e analisar os pacotes um a um.
- **Python:** linguagem usada para escrever o cliente, o servidor e os dois protocolos, usando apenas a biblioteca padrão de *sockets*, sem bibliotecas de rede de alto nível.

3.3. Implementação do Sistema TCP

No modo TCP, o cliente abre uma conexão para cada transferência. Ele primeiro manda uma mensagem do tipo `init` com os dados do arquivo (nome e tamanho), depois envia os blocos um atrás do outro usando `sendall()`, e no final manda uma mensagem `fim`. Como o TCP já é confiável, não foi preciso implementar reenvio nem confirmação, pois o próprio sistema operacional cuida disso.

Um detalhe importante foi a medição do tempo. O comando `sendall()` retorna assim que os dados entram na fila de envio do sistema, e não quando eles chegam de fato ao servidor. Para medir o tempo certo, o cliente, depois de mandar o `fim`, avisa que terminou de enviar com `shutdown(SHUT_WR)` e fica lendo a conexão até o servidor fechá-la. Só nesse momento o cronômetro é parado, garantindo que o tempo medido seja o da entrega completa.

O servidor recebe os blocos, junta tudo em um arquivo e, ao receber o `fim`, calcula o tempo e a vazão daquela transferência.

3.4. Implementação do Sistema R-UDP

O R-UDP foi onde ficou a maior parte do esforço, já que toda a confiabilidade teve que ser feita à mão. A seguir estão os pontos principais.

Estrutura dos pacotes. Toda mensagem é formada por um cabeçalho de texto, no estilo do HTTP, seguido de um corpo (o payload). O cabeçalho é uma lista de campos no formato `Chave: Valor`, separados por `\r\n`, e termina com uma linha em branco (`\r\n\r\n`). Os campos estão descritos na Tabela 2.

Tabela 2. Campos do cabeçalho de aplicação

Campo	Para que serve
X-Custom-Auth	Hash SHA-256 da matrícula e do nome do aluno
X-Type	Tipo da mensagem: <code>init</code> , <code>data</code> , <code>ack</code> ou <code>fim</code>
X-Seq	Número de sequência do bloco
X-Length	Tamanho em bytes do corpo da mensagem
X-Checksum	CRC-32 do bloco, em hexadecimal (só nas mensagens <code>data</code>)
X-Filename	Nome do arquivo (só na mensagem <code>init</code>)
X-Filesize	Tamanho total do arquivo (só na mensagem <code>init</code>)

Número de sequência. Cada bloco recebe um número crescente no campo `X-Seq`. O servidor guarda qual é o número que ele espera receber. Se chegar um bloco com o número esperado, ele é gravado e o servidor passa a esperar o próximo. Se chegar um número repetido (o que acontece quando há reenvio), o servidor não grava de novo, mas confirma mesmo assim, para o cliente parar de reenviar.

ACKs. Para cada mensagem recebida, o servidor responde com um pacote `ack` carregando o mesmo número de sequência. É assim que o cliente sabe que pode seguir em frente.

Timeout e retransmissão. Depois de enviar um pacote, o cliente espera o ACK por no máximo 1 segundo. Se o ACK não chegar nesse tempo, o cliente reenvia o mesmo pacote e espera de novo. Esse reenvio acontece quantas vezes for preciso, sem limite, até

o ACK chegar. Foi essa a escolha feita para garantir que o arquivo sempre seja entregue por completo, mesmo em redes muito ruins.

Checksum. Cada bloco de dados carrega no campo X-Checksum um valor CRC-32 calculado a partir do conteúdo do bloco. Ao receber, o servidor recalcula o CRC-32 e compara com o valor recebido. Se for diferente, o bloco está corrompido e é descartado sem confirmação, o que faz o cliente reenviá-lo pelo timeout.

Campo X-Custom-Auth. Toda mensagem leva o campo X-Custom-Auth, que contém o hash SHA-256 da junção da matrícula com o nome do aluno. Esse campo identifica a origem do tráfego e pode ser visto diretamente no Wireshark, conforme exige o trabalho.

O tamanho de cada bloco foi fixado em 1.300 bytes. Esse valor foi escolhido para que o pacote inteiro (bloco + cabeçalho de aplicação de 147 bytes + cabeçalho UDP de 8 bytes + cabeçalho IP de 20 bytes) fique abaixo do limite de 1.500 bytes da rede Ethernet (a MTU). Assim o pacote não precisa ser quebrado em pedaços pela rede, o que facilita a captura completa no `tcpdump`. Como o arquivo de teste tem 1 MiB (1.048.576 bytes), cada transferência gera 807 blocos de dados.

3.5. Cenários de Teste

A rede foi alterada com o comando `tc qdisc netem` para criar três situações diferentes, descritas na Tabela 3. O atraso e a perda foram aplicados na saída do servidor.

Tabela 3. Cenários de simulação de rede

Cenário	Atraso	Perda	Comando aplicado
A	10 ms	0%	<code>netem delay 10ms loss 0%</code>
B	50 ms	5%	<code>netem delay 50ms loss 5%</code>
C	100 ms	10%	<code>netem delay 100ms loss 10%</code>

O Cenário A representa uma rede boa, quase sem problemas. O Cenário B representa uma rede mediana, com algum atraso e alguma perda. O Cenário C representa uma rede ruim, com atraso alto e bastante perda de pacotes.

3.6. Coleta de Dados

Para cada combinação de protocolo e cenário foram feitas 30 transferências seguidas do mesmo arquivo de 1 MiB, gerado com bytes aleatórios. O número de 30 execuções foi escolhido por estar dentro da faixa pedida pelo trabalho (de 10 a 30) e por dar uma amostra grande o suficiente para deixar as médias mais estáveis.

Em cada transferência o sistema registra o tempo gasto e calcula a vazão (throughput). A vazão é calculada pela fórmula

$$\text{throughput (Mbps)} = \frac{\text{bytes} \times 8}{\text{tempo} \times 1.000.000},$$

ou seja, o tamanho do arquivo em bits dividido pelo tempo em segundos, convertido para megabits por segundo. Vale notar que essa conta usa só os bytes úteis do arquivo, sem contar os cabeçalhos, então o que se mede é a vazão útil.

Ao final das 30 execuções, o sistema calcula e salva em um arquivo CSV a vazão mínima, a vazão média, a vazão máxima e o desvio padrão. O desvio padrão é calculado sobre os valores de vazão e por isso também é dado em Mbps. Ele mostra o quanto os resultados variaram: quanto menor o desvio, mais parecidas foram as execuções entre si.

4. Resultados e Discussão

4.1. Resultados do TCP

A Tabela 4 resume os números do TCP nos três cenários. As Figuras 1, 2 e 3 mostram o tempo médio, a vazão e o desvio padrão do TCP em cada cenário.

Tabela 4. Resultados do TCP (30 execuções, arquivo de 1 MiB)

Cenário	Vazão mín. (Mbps)	Vazão méd. (Mbps)	Vazão máx. (Mbps)	Desvio (Mbps)	Tempo méd. (s)
A	54,23	57,93	58,72	0,77	0,145
B	13,80	23,15	23,83	2,48	0,369
C	8,08	11,57	11,95	1,11	0,734

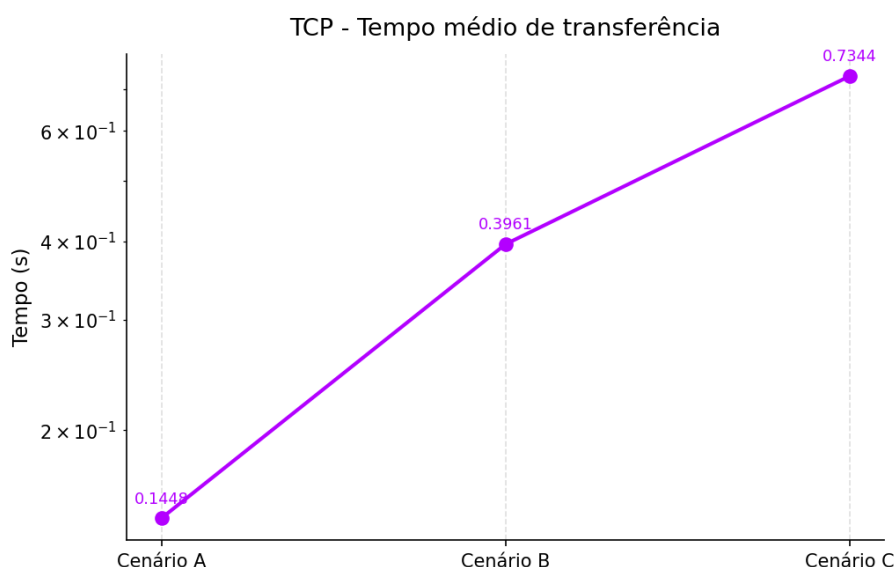


Figura 1. Tempo médio de transferência do TCP por cenário.

O TCP se manteve rápido em todos os cenários. No Cenário A, a vazão ficou perto dos 58 Mbps. Conforme o atraso e a perda aumentaram, a vazão caiu, mas continuou alta: cerca de 23 Mbps no B e 12 Mbps no C. Isso mostra que o TCP lida bem com perdas, pois ele reenvia rápido e tem uma janela que permite mandar vários pacotes de uma vez.

Um ponto interessante são algumas execuções que destoaram das demais. No Cenário B, duas execuções caíram para cerca de 14 Mbps, e no Cenário C, três execuções caíram para cerca de 8 Mbps. Esses casos acontecem quando o TCP precisa esperar um tempo maior para reenviar um pacote perdido (o chamado *timeout* de retransmissão) e diminui a janela por causa do controle de congestionamento. A quantidade desses casos bate com a perda configurada: 2 em 30 no Cenário B (perto de 5%) e 3 em 30 no Cenário C (10%).

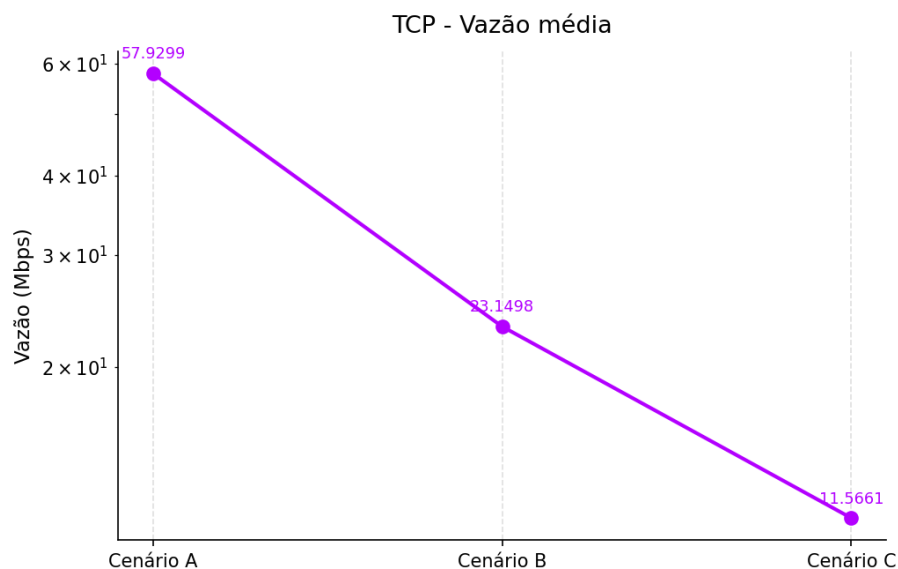


Figura 2. Vazão média do TCP por cenário.

4.2. Resultados do R-UDP

A Tabela 5 resume os números do R-UDP, e as Figuras 4, 5 e 6 mostram o tempo médio, a vazão e o desvio padrão em cada cenário. Repare que a escala de vazão aqui é muito menor que a do TCP.

Tabela 5. Resultados do R-UDP (30 execuções, arquivo de 1 MiB)

Cenário	Vazão mín. (Mbps)	Vazão méd. (Mbps)	Vazão máx. (Mbps)	Desvio (Mbps)	Tempo méd. (s)
A	0,9741	0,9756	0,9773	0,0006	8,60
B	0,0882	0,1032	0,1197	0,0082	81,79
C	0,0440	0,0489	0,0532	0,0023	171,87

O R-UDP entregou todos os arquivos corretamente em todos os cenários, ou seja, a parte de confiabilidade funcionou. Porém a vazão foi muito baixa. Mesmo no Cenário A, sem nenhuma perda, a vazão ficou em cerca de 0,98 Mbps, e o tempo de cada transferência foi de 8,6 segundos.

O motivo é o Stop-and-Wait. Como o cliente manda um bloco e fica parado esperando a confirmação antes de mandar o próximo, ele gasta um tempo de ida e volta (RTT) inteiro para cada bloco. São 807 blocos, então no Cenário A são 807 vezes cerca de 10 ms, o que dá perto de 8 segundos, justamente o que foi medido.

Nos cenários com perda a situação piora muito, porque cada confirmação perdida custa 1 segundo de espera (o timeout) antes do reenvio. No Cenário C, com 10% de perda, o tempo médio passou de 171 segundos. O desvio padrão do R-UDP foi pequeno, o que mostra que, apesar de lento, o comportamento foi bem previsível: no Cenário A o desvio foi quase zero, porque sem perda todas as execuções são praticamente iguais.

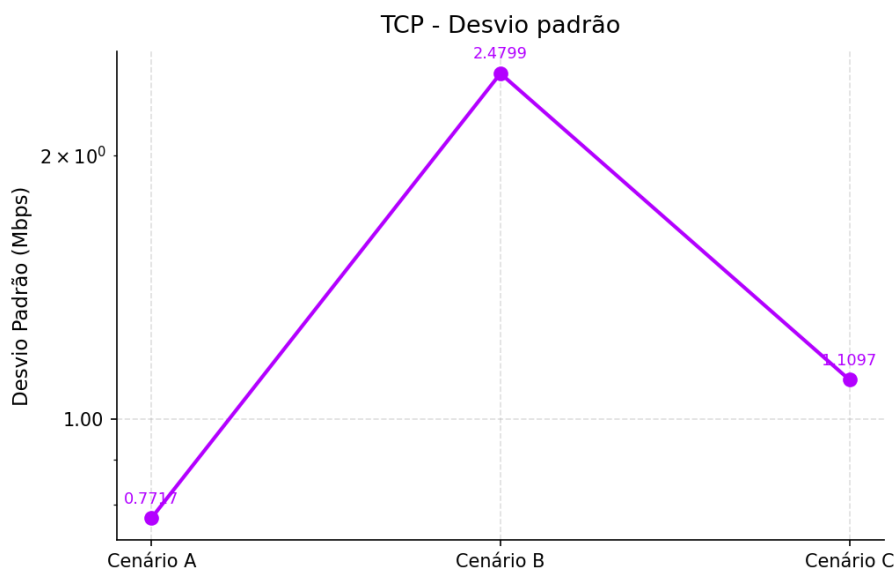


Figura 3. Desvio padrão da vazão do TCP por cenário.

4.3. Comparação entre TCP e R-UDP

As Figuras 7, 8 e 9 colocam os dois protocolos lado a lado, comparando tempo, vazão e desvio padrão em cada cenário. As escalas são logarítmicas, pois os valores de TCP e R-UDP são muito diferentes entre si.

Comparação dos tempos. A diferença de tempo é enorme. No Cenário A, o TCP levou 0,145 segundo e o R-UDP levou 8,6 segundos. No Cenário C, o TCP levou menos de 1 segundo e o R-UDP levou quase 3 minutos. Quanto pior a rede, maior fica a distância entre os dois, como mostra a Figura 7.

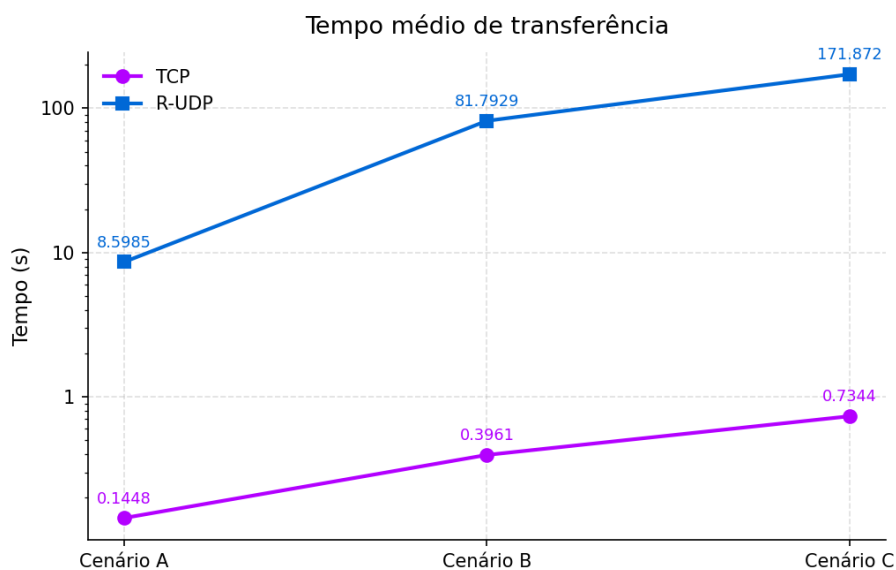


Figura 7. Comparação do tempo médio de transferência entre TCP e R-UDP.

Comparação da vazão. A Tabela 6 mostra quantas vezes o TCP foi mais rápido

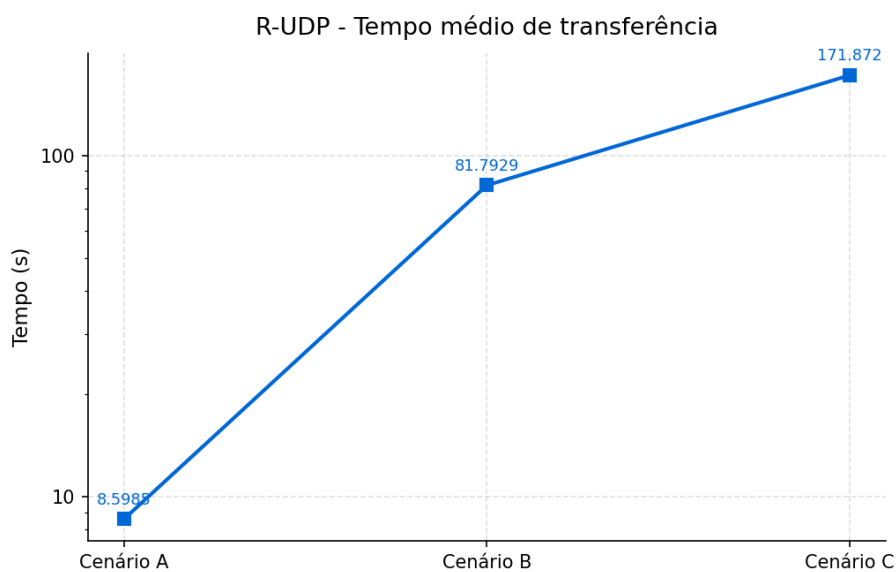


Figura 4. Tempo médio de transferência do R-UDP por cenário.

que o R-UDP em cada cenário. A vantagem do TCP cresce conforme a rede piora, indo de cerca de 59 vezes no Cenário A para cerca de 237 vezes no Cenário C, o que aparece também na Figura 8.

Tabela 6. Quantas vezes a vazão do TCP foi maior que a do R-UDP

Cenário	TCP (Mbps)	R-UDP (Mbps)	Vantagem do TCP
A	57,93	0,9756	59 vezes
B	23,15	0,1032	224 vezes
C	11,57	0,0489	237 vezes

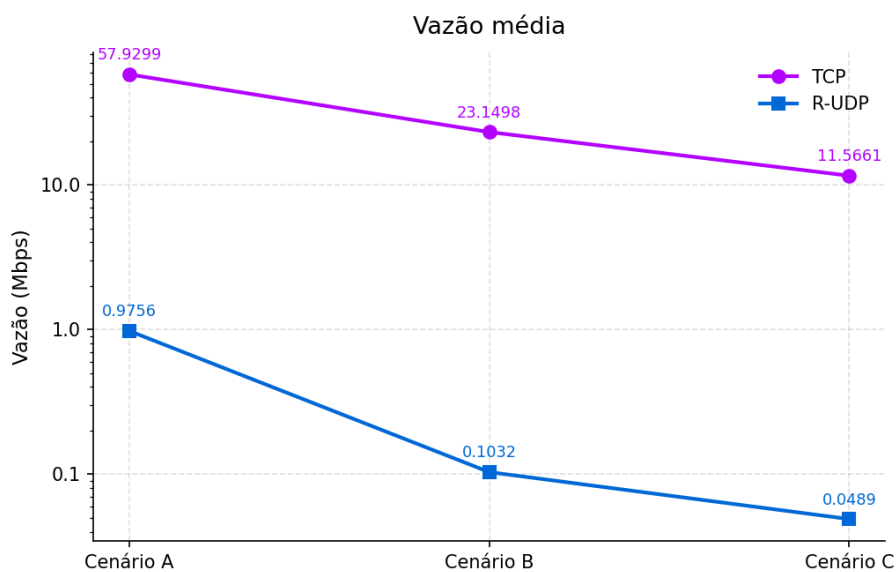


Figura 8. Comparação da vazão média entre TCP e R-UDP.

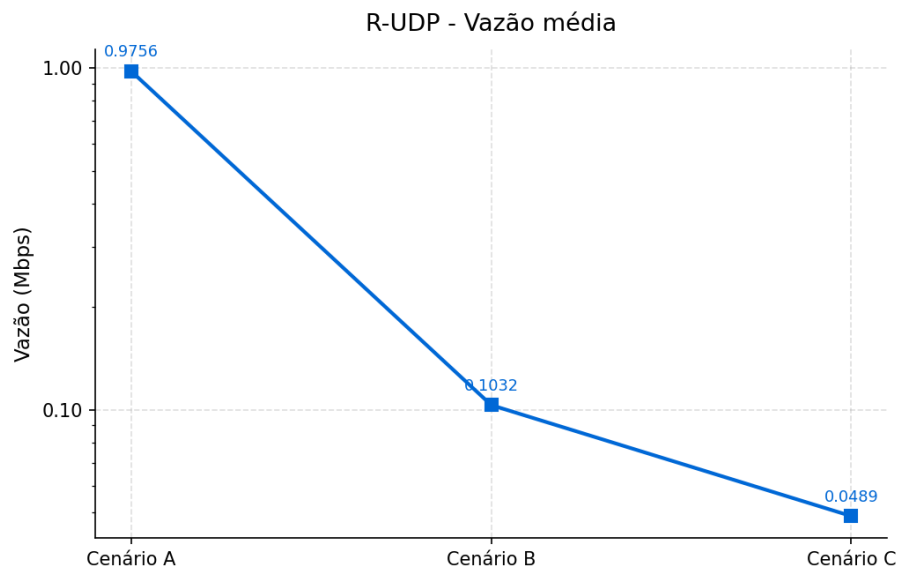


Figura 5. Vazão média do R-UDP por cenário.

Comparação da estabilidade. Olhando o desvio padrão em relação à média, o TCP variou mais que o R-UDP, como mostra a Figura 9. Isso acontece porque o TCP é rápido e, quando ocorre uma perda que dispara um timeout maior, a queda na vazão é grande em valor absoluto. Já o R-UDP é tão lento e regular que uma perda a mais ou a menos quase não muda o resultado final, deixando suas execuções mais parecidas entre si.

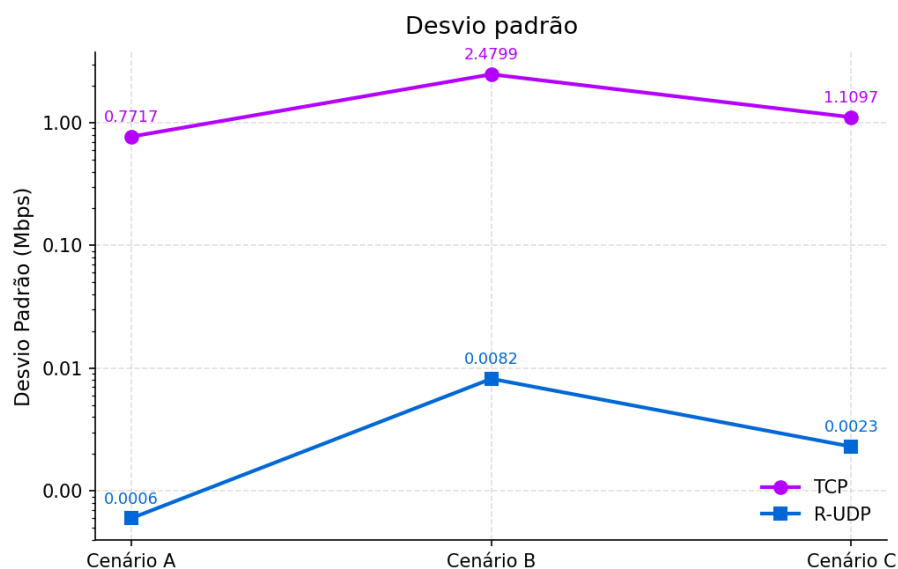


Figura 9. Comparação do desvio padrão da vazão entre TCP e R-UDP.

4.4. Validação Cruzada com Wireshark/TCPDump

Durante todos os testes, o tcpdump capturou o tráfego da rede e salvou em arquivos .pcap, um para cada cenário. Abrindo esses arquivos no Wireshark, foi possível confirmar três coisas.

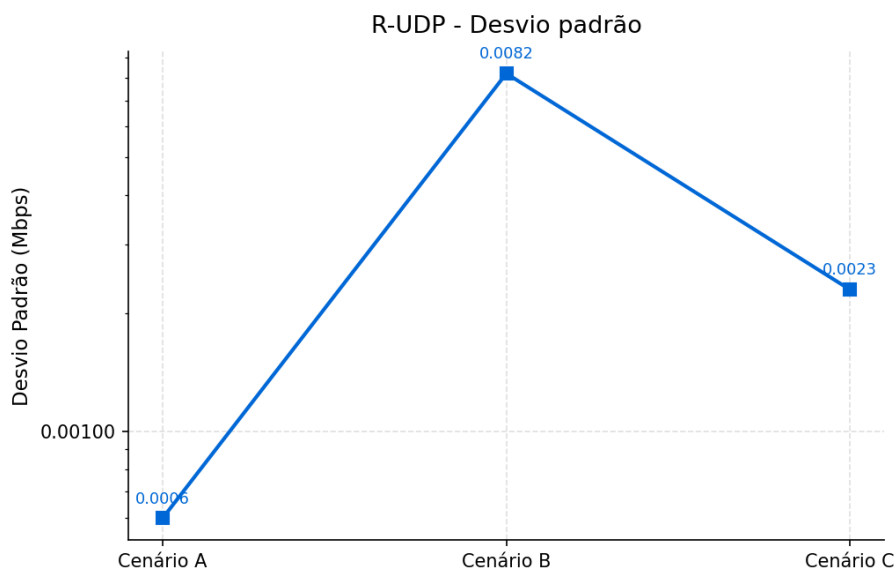


Figura 6. Desvio padrão da vazão do R-UDP por cenário.

Identificação do X-Custom-Auth. Ao abrir um pacote no Wireshark e olhar o conteúdo, o campo X-Custom-Auth aparece em texto, com o hash SHA-256 da identidade. Isso prova que o tráfego analisado é mesmo o do sistema desenvolvido.

Verificação dos pacotes e da perda. No R-UDP foi possível ver pacotes com o mesmo número de sequência aparecendo duas vezes, separados por exatamente 1 segundo. Esses são os reenvios causados pela perda das confirmações, e a distância de 1 segundo entre eles é o tempo de timeout configurado. A Tabela 7 mostra a quantidade de pacotes capturados em cada arquivo.

Tabela 7. Pacotes capturados pelo tcpdump (30 execuções por cenário)

Cenário	TCP		R-UDP	
	Pacotes	Tamanho (MB)	Pacotes	Tamanho (MB)
A	29.055	34,7	48.535	38,4
B	10.629	33,4	49.744	40,1
C	15.284	33,9	51.242	42,3

Comparação entre a aplicação e a rede. Os números da captura batem com o que a aplicação fez. No R-UDP do Cenário A, sem perda, foram capturados 48.535 pacotes, perto do dobro dos 24.210 blocos de dados das 30 execuções (807 blocos vezes 30), pois para cada bloco existe uma confirmação de volta. Nos cenários com perda, sobram pacotes a mais, que são justamente os reenvios. Comparando com o Cenário A, o Cenário B teve cerca de 1.209 pacotes a mais (perto dos 5% de perda) e o Cenário C teve cerca de 2.707 pacotes a mais (perto dos 10% de perda). Ou seja, a quantidade de reenvios vista na rede confirma a taxa de perda que foi configurada.

Comparação dos tempos. Além do volume, o tempo também foi conferido. Cada execução foi isolada dentro do .pcap usando o pacote `init` como início, medindo até o último pacote daquela execução. A Tabela 8 compara o tempo médio medido pela aplicação em Python com o tempo médio observado na captura, nos seis casos.

Tabela 8. Tempo médio por execução: aplicação (Python) versus captura (tcp-dump)

Cenário	Protocolo	Aplicação (s)	Pcap (s)	Diferença
A	TCP	0,1448	0,1543	+6,50%
B	TCP	0,3691	0,4165	+12,83%
C	TCP	0,7344	0,9304	+26,70%
A	R-UDP	8,5985	8,5960	-0,03%
B	R-UDP	81,7929	81,7718	-0,03%
C	R-UDP	171,8716	171,8538	-0,01%

No R-UDP os tempos batem quase perfeitamente, com diferença abaixo de 0,03%, porque o trecho capturado vai do `init` até o último ACK, que é exatamente o que a aplicação cronometra. No TCP a captura mostra um tempo um pouco maior, e essa diferença cresce com a latência, indo de 6,5% no Cenário A para 26,7% no Cenário C. Isso acontece porque o trecho capturado inclui a abertura e o fechamento da conexão TCP (os pacotes SYN e FIN), que a aplicação não cronometra por completo e cujo custo aumenta junto com o atraso da rede.

4.5. Respostas às Perguntas

1. Como a curva de throughput do TCP se comportou em comparação ao R-UDP sob 10% de perda?

No Cenário C, que tem 10% de perda, o TCP manteve uma vazão média de 11,57 Mbps, enquanto o R-UDP ficou em apenas 0,0489 Mbps, ou seja, o TCP foi cerca de 237 vezes mais rápido. A curva do TCP ficou quase reta, perto dos 12 Mbps, com poucas quedas para cerca de 8 Mbps quando havia reenvios. A curva do R-UDP também ficou baixa e regular, sempre perto de 0,05 Mbps. A grande diferença se explica pelo método de cada um: o TCP envia vários pacotes antes de esperar confirmação e reenvia rápido, enquanto o R-UDP, no Stop-and-Wait, espera 1 segundo inteiro a cada confirmação perdida, o que derruba muito o desempenho.

2. Qual foi o overhead (em bytes) introduzido pelos cabeçalhos do protocolo R-UDP?

Cada pacote de dados do R-UDP tem um cabeçalho de aplicação de 147 bytes, somado a um bloco de 1.300 bytes, o que dá um pacote de 1.447 bytes. Assim, o cabeçalho representa cerca de 10,16% de cada pacote de dados. Somando também os cabeçalhos do UDP (8 bytes) e do IP (20 bytes), o pacote vai para 1.475 bytes na rede e o overhead total sobe para cerca de 11,86%. Além disso, cada pacote de confirmação tem 121 bytes e é totalmente sobrecarga, pois não carrega dados do arquivo. Em uma transferência de 1 MiB, sem contar reenvios, isso dá perto de 118.629 bytes só de cabeçalhos dos blocos mais cerca de 97.647 bytes de ACKs, totalizando perto de 216 KB de sobrecarga para entregar 1 MB de dados úteis.

3. Houve diferença entre o tempo medido pelo Python e o registrado no Wireshark/TCPDump? Justifique.

Sim, houve uma pequena diferença, medida na Tabela 8. No R-UDP o tempo da aplicação e o da captura são praticamente iguais, com diferença abaixo de 0,03%. No

TCP a captura registra um tempo um pouco maior, indo de 6,5% no Cenário A a 26,7% no Cenário C, conforme a latência aumenta. A explicação é que o cronômetro do Python mede apenas o envio dos dados da aplicação, enquanto a captura na placa de rede inclui também a abertura e o fechamento da conexão TCP (os pacotes SYN e FIN), cujo custo cresce com o atraso da rede. No R-UDP, que não tem essa abertura e fechamento de conexão, os dois tempos coincidem. Portanto, as diferenças são pequenas, explicadas pelo funcionamento de cada protocolo, e não indicam erro de medição.

5. Conclusão

Este trabalho mostrou, na prática, a diferença entre usar um protocolo confiável pronto (o TCP) e construir a confiabilidade na mão sobre o UDP (o R-UDP). Os dois sistemas entregaram os arquivos corretamente em todos os cenários, então ambos cumpriram o objetivo de transferir os dados sem erros.

A diferença apareceu no desempenho. O TCP foi muito mais rápido, de 59 a 237 vezes, dependendo do cenário, e essa vantagem aumentou conforme a rede piorou. A validação com o Wireshark confirmou que os dados da aplicação batem com o tráfego real, inclusive a quantidade de reenvios, que ficou igual à taxa de perda configurada no `tc`.

A principal limitação está no método Stop-and-Wait usado no R-UDP. Por enviar um pacote de cada vez e esperar a confirmação antes de continuar, ele desperdiça quase todo o tempo apenas esperando, o que explica a vazão muito baixa, mesmo numa rede perfeita. Métodos com janela, como o Go-Back-N e o Selective Repeat, enviariam vários pacotes de uma vez e teriam um desempenho bem melhor.

O impacto da perda e do atraso ficou muito claro no R-UDP. Como cada confirmação perdida custa 1 segundo de espera, a perda de pacotes pesa muito mais nele do que no TCP, que reage de forma mais rápida e inteligente. Outra limitação foi o ambiente de teste: o atraso e a perda foram aplicados só na saída do servidor, então a perda atingiu principalmente as confirmações, e não os dados de ida. Em uma rede real, a perda aconteceria nos dois sentidos.